

AI와 함께 에이전트를 만든 이야기

메시지 알림 라우터가 무엇인지보다, Claude와 어떻게 협업해 그것을 만들었는지에 초점을 맞춘 발표. 표준 AI 에이전트 아키텍처와 실제 우리가 만든 시스템을 비교하고, 8번의 반복에서 사람과 Claude가 실제로 주고받은 판단의 흐름을 재구성한다.

8번의 반복

표준 아키텍처 대응·차이

협업 흐름 재구성

프로젝트 결과보다, 협업 과정

간단히만 다룬다

- 메시지 알림 라우터가 정확히 무엇을 하는 시스템인지
- 파이프라인의 세부 구현(캐싱, 재시도 등)
- 최종 정확도 수치 — 부록으로만 언급

핵심으로 다룬다

- Claude와 실제로 어떤 판단·대화가 오갔는가
- 표준 AI 에이전트 아키텍처와 비교했을 때 무엇이 대응되고, 무엇이 다르거나 추가됐는가
- 8번의 피드백 루프에서 사람과 Claude가 각각 무엇을 말았는가

무엇을 만들었나 (압축)

WhatsApp 메시지를 수신자별로 notify(즉시)/digest(나중에)/mute(억제)로 개인화 라우팅하는 시스템.

로컬 전처리 → 컨텍스트 조립 → Claude Sonnet 5 → 검증·fallback → output.csv

이 파이프라인 자체보다 — 이걸 어떻게 Claude와 함께 설계했는지가 이 발표의 본론이다.

표준 AI 에이전트 아키텍처

널리 쓰이는 LLM 에이전트 설계를 6개 요소로 정리하면 대략 이렇다 — 이후 슬라이드에서 우리 시스템과 하나씩 대응시킨다.

1

지각 (Perception)

원시 입력(텍스트·이미지·음성 등)을 모델이 다룰 수 있는 형태로 정규화

2

기억 (Memory)

단기 작업 컨텍스트 + 장기/외부 지식에 대한 검색

3

계획·추론 (Planning / Reasoning)

목표를 하위 작업으로 분해하고, 사고 과정을 명시화

4

도구 사용·행동 (Tool Use / Action)

외부 API·DB·코드 실행으로 환경에 직접 개입

5

관찰 → 재계획 (Observe → Replan)

행동 결과를 보고 다음 스텝을 결정, 목표 달성까지 반복하는 루프

6

출력·자기평가 (Output / Self-eval)

최종 산출물 생성 + 피드백으로 스스로 개선

표준 요소 ↔ 우리가 만든 시스템

표준 요소	우리 시스템의 대응	비고
1. 지각	image_pipeline.py / audio_pipeline.py — OCR·설명, 음성 전사	표준과 거의 동일
2. 기억	db.py(SQLite) + pull_context()의 규칙 기반 검색(발신자>그룹>비즈니스>최신순)	임베딩 대신 설명 가능한 휴리스틱으로 대체 — 데이터 규모가 작아 충분
3. 계획·추론	내부 스키마 필드(sender_trust 등)를 최종 판단 전에 강제로 채움	자유형식 Thought 대신 카테고리형 필드로 구조화
4. 도구 사용	없음 — LLM은 이미 조립된 컨텍스트에 대해 1회 분류만 수행	가장 큰 차이 — 다음 슬라이드
5. 관찰→재계획	런타임엔 없음(재시도만); 개발 단계에서 사람+Claude가 8회 수행	루프의 위치가 inference-time이 아니라 build-time
6. 출력·자기평가	to_output_row() + code/evaluation/ 자동 채점 하네스	대응되지만 오프라인 배치 평가

런타임에 자율 도구 호출 루프가 없다

표준 에이전트는 실행 중 LLM이 스스로 어떤 도구를 쓸지 고르고, 결과를 보고, 다시 판단하는 다단계 루프(ReAct류)를 돈다. 우리 라우터는 그렇게 하지 않았다 — 도구 호출(DB 조회, 미디어 분석)을 LLM 호출 이전에 결정론적 파이프라인이 전부 끝내고, LLM은 이미 조립된 컨텍스트에 대해 딱 한 번, JSON 스키마로 제약된 분류만 한다.

왜 자유도를 뺐는가: 같은 입력엔 같은 종류의 근거로 판단해야 하는 신뢰성, 무엇을 보고 판단했는지 항상 재현 가능해야 하는 평가 가능성, 메시지당 여러 번의 도구 호출 대신 파이프라인 1회+LLM 호출 1회로 끝내는 비용/지연, 그리고 결정론적 동작을 요구하는 제출 규약 — 이 네 가지 때문에 의도적으로 자유 루프를 배제했다.

진짜 에이전트 루프는 다른 곳에 있었다

표준 아키텍처 5번, 관찰 → 재계획 루프가 사라진 게 아니라 — 런타임이 아니라 개발 타임으로 옮겨갔다.

사람이 결과를 관찰하고 → 무엇을 할지 계획하면 → Claude Code가 원인을 조사하고 실행하고 → 평가 하네스로 검증하고 → 반영하거나 롤백한다. 이 루프를 8번 돌렸다.

다음 슬라이드들은 CLAUDE.ko.md에 기록된 실제 반복·의사결정 로그를 사람-Claude 대화 형식으로 재구성한 것이며, 원문 채팅 로그 그대로의 캡처는 아니다.

버그를 스스로 발견하고 고치다

사람

"route --limit 3로 먼저 작게 테스트해보자."

Claude

실행 결과 0/3, 그것도 비정상적으로 빨리 끝난 걸 스스로 이상하게 여김 → 로그 확인 → ThreadPoolExecutor 워커들이 SQLite 커넥션 하나를 공유하고 있던 게 원인임을 특정 → 워커마다 자기 커넥션을 열도록 수정 → 재실행 3/3 성공을 보고.

사람은 "작게 먼저 테스트하자"는 방향만 줬고, 원인 조사부터 수정까지는 Claude가 스스로 수행했다.

채점이 이상하다는 걸 스스로 알아채다

사람

"30건 정답 세트 기준으로 채점해줘."

Claude

채점 결과가 0/30으로 나오는 걸 이상하게 여김 → id 체계를 조사해 sample_messages.csv(sample_msg_*)와 messages.csv(msg_*)가 서로 다른 네임스페이스임을 발견 → 채점 스크립트를 sample_messages.csv 자신의 입력으로 라우터를 직접 돌리는 방식으로 재작성 → 기준선 action 90% · type 63% · F1 49%를 보고.

겉보기엔 사소한 채점 스크립트 문제였지만, "결과가 이상하다"를 감지하고 끝까지 원인을 추적한 건 Claude였다.

사람이 방향을 주면, Claude가 근거를 찾는다

사람

"message_type 정확도(63%)가 낮는데, 이거 개선해줘."

Claude

혼동행렬을 만들어 오답이 어디 몰려있는지 확인 → event / promotion / greeting 카테고리의 실제 오답 텍스트를 전부 정독 → 이 데이터셋만의 분류 관습 4가지를 규칙으로 도출·반영 → 63%→90% 개선을 보고.

사람은 "무엇을" 개선할지 지정했고, "어떻게"(원인 진단과 일반화 가능한 규칙 도출)는 Claude의 몫이었다.

사람이 패턴을 보고, Claude가 구조로 만든다

사람

“정답이 digest인데 notify로 틀린 경우가 많고, 틀린 답들의 confidence가 낮아 보여 — 이거 활용할 수 있을까?”

Claude

실측으로 확인(정답 평균 confidence 0.85 vs 오답 0.70) → 최종 판단 전에 반드시 채워야 하는 내부 전용 스키마 필드 5개 설계·구현 + “애매하면 digest 기본값” 규칙 추가 → 이전 오답 7건만 먼저 저비용 검증(3건 수정 확인) → 30건 전체로 재확인 → 86.67%→93.33%를 보고.

사람의 관찰(패턴)을 Claude가 구체적인 스키마 설계로 번역했다.

실패를 스스로 인정하고 되돌리다

사람

"type_candidates에 각 후보를 뒷받침하는 실제 문구(key_phrase)도 추출하게 해보면 어떨까?"

Claude

구현 → 30건으로 검증 → 93.33%→86.67%로 퇴보한 것을 스스로 확인 → 개선이 아니라 퇴보라고 먼저 보고하고, 반복 7 상태로 즉시 되돌림 → 110건 재분류.

그렇듯한 제안도 측정 없이 채택하지 않는다 — Claude가 스스로 반박 근거를 찾아 되돌린 사례.

사람이 한 일 / Claude가 한 일

사람이 한 일

- 결과를 직접 읽고 패턴을 발견 (digest→notify 편향, confidence-정확도 상관관계)
- 개선 방향과 우선순위 제시 ("message_type 정확도부터 고쳐줘")
- 제안된 다음 아이디어를 승인 (key_phrase 확장)

Claude가 한 일

- 버그·이상 신호를 스스로 감지하고 원인을 끝까지 조사
- 가설을 구조(스키마·규칙)로 구현하고 측정으로 검증
- 퇴보를 스스로 확인하고 먼저 보고한 뒤 즉시 롤백

이 협업에서 배운 것

가장 중요한 산출물은 output.csv가 아니라, “관찰→가설→구현→검증→반영or롤백”을 8번 반복한 개발 루프 그 자체였다.

표준 에이전트 아키텍처가 설명하는 자율 루프를, 이 프로젝트는 런타임 대신 Claude Code와의 협업이라는 형태로 개발 타임에 구현한 셈이다.

93.33%

action 정확도 (부록)

93.33%

message_type 정확도 (부록)

110/110

output.csv 행 (부록)